

Personalized E-Commerce Search

ML System Design Assignment

Overview

This is a system design assignment. You will design a way to personalize product search results using signals from other users' order history and search queries, layered on top of an existing keyword search system.

We are evaluating how you scope an ambiguous problem, the trade-offs you make, and how clearly you communicate them. A simple, well-reasoned design beats an elaborate one. If you find yourself reaching for a sophisticated technique, first ask whether a simpler heuristic gets you 80% of the value — and say so in your write-up.

Current State

Search today works purely on keyword matching. Product titles and descriptions are indexed in an inverted-index engine (think Elasticsearch/Solr) with BM25/TF-IDF style relevance scoring. When a user types a query, the engine returns products ranked by textual relevance to that query alone.

The system has no notion of who is searching. Two different users typing the identical query see identical results. No purchase history, click behavior, or population-level signal is used anywhere in ranking.

Data & Assumptions

Assume the following is available to you. Where the brief is silent on something you need, state your own assumption explicitly and briefly justify it — don't block on it.

Source	Fields (illustrative)	Volume
Product catalog	product_id, title, description, category, brand, price, attributes	~5M SKUs
Search logs	user/session_id, raw query text, timestamp, results shown, click position, click/purchase flag	~200M searches/month
Order history	user_id, product_id, order timestamp, query that led to order (when known)	~5M orders/month

- **Users:** a mix of logged-in users (persistent user_id) and anonymous/guest sessions (device or session id only).
- **ML infra:** assume none exists today beyond basic batch analytics jobs. This would be the first personalization system introduced into this stack.

Your Task

Address both parts below. Part 1 is the modeling problem; Part 2 is making it real inside the current engineering system.

Part 1 — Personalization / Recommendation Engine

Design how order history and other users' search queries turn into personalized results for a given searcher's keyword query. Some pointers(choose 0 or more from below):

- Which signals you'd use from order history and search logs, and how you'd turn raw logs into something usable (e.g., query → co-purchased/co-searched products, user → affinity categories).
- How a user's current keyword query connects to relevant historical behavior — from other users as well as their own.
- Cold start: brand-new users with no history, and brand-new products with no purchase/search signal yet.
- What can be precomputed offline (batch) versus what must be computed at request time, and why.
- How you'd know it's working: offline metrics to validate before launch, and online metrics/A-B design to confirm impact after launch.
- Any more sophisticated alternative you considered and deliberately rejected (e.g., a learned ranking model, embedding-based retrieval) — explain why it wasn't worth the complexity at this stage.

Part 2 — Integration into the Existing Engineering System

Design how this fits into the keyword search system described above with minimal disruption. Some pointers(choose 0 or more from below):

- Where personalization logic lives relative to keyword retrieval: influencing the query itself, re-ranking after retrieval, or blending a separate candidate list.
- What new components are needed — batch jobs, a feature/lookup store, a re-ranking service — and how they talk to the existing search API.
- How you stay within the current latency budget: what's cached, what's precomputed, what's looked up online.
- Fallback behavior when personalization data is missing, stale, or the service is slow/down — the system should always degrade gracefully to today's keyword search, never fail the search request.
- A rollout plan: how you'd ship this safely (e.g., shadow traffic, gradual A/B ramp) rather than flipping it on for everyone at once.
- What you'd monitor post-launch to catch regressions in relevance or latency.

Guidelines

We are deliberately not asking for a deep-learning ranking model, real-time streaming infrastructure, or a multi-stage retrieval funnel unless you can clearly justify why the added complexity is necessary at this scale. A design built on simple aggregation tables (e.g., "query

→ top co-purchased products”) plus a lightweight re-ranking step is a completely acceptable — even preferred — answer, provided you’ve reasoned through it well.

Simple solutions beat complex, over-engineered ones. If two designs solve the problem equally well, pick the simpler one and say why.

AI Tool Usage

You’re welcome to use AI coding or design assistants (ChatGPT, Claude, Copilot, Cursor, etc.) while working on this. If you do, please show us prompts and chat history alongside your submission. This isn’t a restriction on using these tools — we want to understand how you used them and how you evaluated or edited what they produced.